Diseño digital avanzado

Trabajo práctico 1: shift register de LEDs

Krede, Julian

27 de octubre de 2026

Se describe el diseño de un shift register circular que enciende los LEDs de la placa Arty A7 de a uno por vez, con velocidad, habilitación y color seleccionables mediante switches. El diseño incluye un VIO para manejar las entradas en forma remota y un ILA para observar las salidas. Se verificó por simulación y se implementó en una FPGA del cluster remoto.

1. Diseño

El sistema enciende un LED a la vez y desplaza el bit encendido en forma circular (0001 → 0010 → 0100 → 1000 → 0001 . . .). Un contador marca cada cuánto se desplaza. Las entradas son:

- `i_reset`: reset del sistema (botón `ck_rst`, activo en bajo). Pone a cero el contador e inicializa el shift register en `0001`.
- `i_sw[0]`: habilitación. Con 0 el sistema se detiene sin perder el estado del contador ni del shift register.
- `i_sw[2:1]`: selección del límite del contador R0–R3, es decir, de la velocidad.
- `i_sw[3]`: color de los LEDs RGB (0 azul, 1 verde).

La Figura 1 muestra la arquitectura del módulo `top`, que se dividió en dos módulos (`count` y `shiftreg`) más la lógica de selección de entradas y de color.

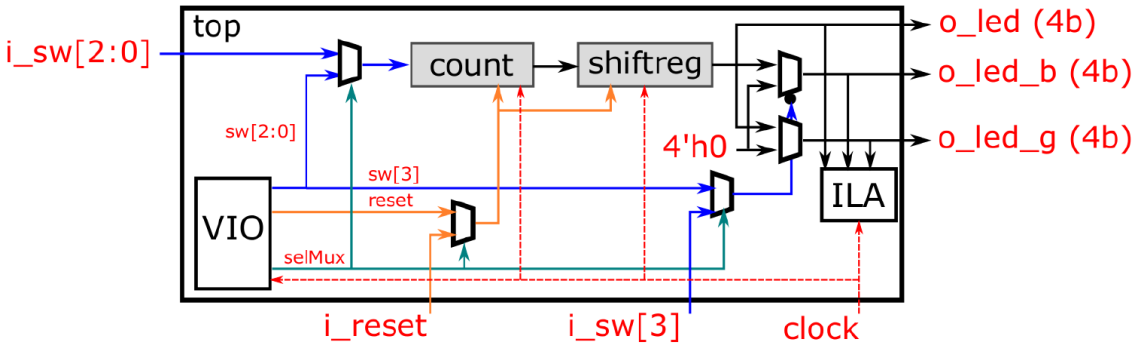


Figura 1: Diagrama en bloques del módulo `top`.

1.1. Contador (`count.v`)

Es un contador de `NB_COUNTER` = 32 bits con reset asincrónico activo en alto. Un multiplexor elige el límite según `i_sel` (`i_sw[2:1]`):

$$R_k = 2^{N_c-10+k} - 1, \qquad k = 0, \ldots, 3,$$

donde N_c es la cantidad de bits del contador. En cada flanco, si `i_enable = 1`:

- si el contador llegó al límite (`counter >= ref_limit`) vuelve a cero;
- si no, se incrementa en uno.

Con `i_enable = 0` mantiene el valor. La salida

```
o_valid = i_enable & (counter >= ref_limit)
```

vale 1 durante el ciclo en que el contador está en el límite y es la que habilita el desplazamiento.

Se comparó con `>=` en lugar de `==` porque el límite puede cambiarse en cualquier momento. Si el contador está, por ejemplo, en 2^{24} y se pasa a R0 ($2^{22} - 1$), con `==` tendría que dar la vuelta completa de 2^{32} cuentas (unos 43 s) hasta volver a pasar por el límite. Con `>=`, en cambio, se reinicia en el ciclo siguiente.

Tiempo entre desplazamientos. El contador recorre los valores $0, 1, \ldots, R_k$ y vuelve a 0, así que hay un desplazamiento cada $R_k+1 = 2^{N_c-10+k}$ ciclos. Con el reloj de 100 MHz ($T_{clk} = 10\,\text{ns}$) y $N_c = 32$:

$$T_k = (R_k + 1)\, T_{clk} = 2^{22+k} \cdot 10\,\text{ns}.$$

`i_sw[2:1]`	Límite	Ciclos por paso	Tiempo por paso
`00`	$R_0 = 2^{22} - 1$	2^{22}	$\approx 42\,\text{ms}$
`01`	$R_1 = 2^{23} - 1$	2^{23}	$\approx 84\,\text{ms}$
`10`	$R_2 = 2^{24} - 1$	2^{24}	$\approx 168\,\text{ms}$
`11`	$R_3 = 2^{25} - 1$	2^{25}	$\approx 336\,\text{ms}$

Cada código de `i_sw[2:1]` duplica el tiempo del anterior. El límite más grande necesita 25 bits, así que el contador de 32 bits alcanza con margen.

1.2. Shift register (`shiftreg.v`)

Es un registro de `N_LEDS` = 4 bits con reset asincrónico activo en alto, que lo inicializa en `0001`. Cuando `i_valid = 1` rota un lugar a la izquierda, y el bit más significativo vuelve a la posición 0:

$$\texttt{shiftreg} \leftarrow \{\texttt{shiftreg[2:0]},\ \texttt{shiftreg[3]}\}.$$

Como `i_valid` es el `o_valid` del contador, el registro se desplaza una sola vez cada T_k, y se queda quieto cuando el sistema está deshabilitado.

1.3. Módulo principal (`top.v`)

- **Selección placa / VIO.** La salida `selMux` del VIO elige de dónde se toman los switches y el reset: de los pines de la placa (`selMux = 0`) o de las salidas del VIO (`selMux = 1`). Esto permite manejar el diseño sin acceso físico a la placa.

- **Reset.** Tanto `ck_rst` como el reset del VIO son activos en bajo; en el `top` se invierten para generar el `reset` interno activo en alto que usan `count` y `shiftreg`.
- **Color.** Dos multiplexores mandan el estado del shift register al canal azul (`o_led_b`) si `sw[3] = 0`, o al verde (`o_led_g`) si `sw[3] = 1`; el otro canal recibe `4'h0`. Los LEDs simples `o_led` siempre muestran el estado.
- **Debug.** El VIO y el ILA observan las señales internas `connect_leds`, `led_b` y `led_g`, ya que no es posible conectar un IP a un puerto de salida.

1.4. Constraints

El archivo `Arty_Master_v2.xdc` asigna los pines de la placa: el reloj de 100 MHz (`E3`) con su `create_clock` de 10 ns, los switches SW0–SW3 a `i_sw[3:0]`, el botón `ck_rst` a `i_reset`, los canales azul y verde de los LEDs RGB LD0–LD3 a `o_led_b` y `o_led_g`, y los LEDs LD4–LD7 a `o_led`.

2. Simulación

El testbench `tb_shiftleds.v` instancia el `top` sin los IP cores, como pide el enunciado. Como en simulación nadie maneja el VIO, se fuerza `selMux = 0` con `force` para que las entradas se tomen de la placa.

Para que la simulación no tarde demasiado, el contador se parametrizó con `NB_COUNT = 14`. Con la fórmula anterior los límites quedan

$$R_k + 1 = 2^{14-10+k} = 2^{4+k} = 16,\ 32,\ 64,\ 128 \text{ ciclos} \quad \Rightarrow \quad T_k = 160,\ 320,\ 640,\ 1280 \text{ ns}.$$

Secuencia de estímulos:

1. Reset activo durante 100 ns y luego se libera.
2. Se habilita el sistema (`sw[0] = 1`) con `sw[2:1] = 00`.
3. Cada 1 ms se cambia la configuración: `sw[2:1] = 01`, luego `10`, luego `sw[3] = 1` (verde) y por último `sw[2:1] = 11`.

2.1. Resultados

Las siguientes capturas son del simulador de Vivado. La señal `i_sw` se muestra como {`sw[3], sw[2], sw[1], sw[0]`}, así que, por ejemplo, `0011` significa sistema habilitado con `sw[2:1] = 01` y color azul.

Reset. Mientras `i_reset` está en 0 (activo en bajo), el shift register queda en `0001`, el contador en 0, y como `sw[3] = 0` el estado se ve en el canal azul (Figura 2). En 100 ns se libera el reset.

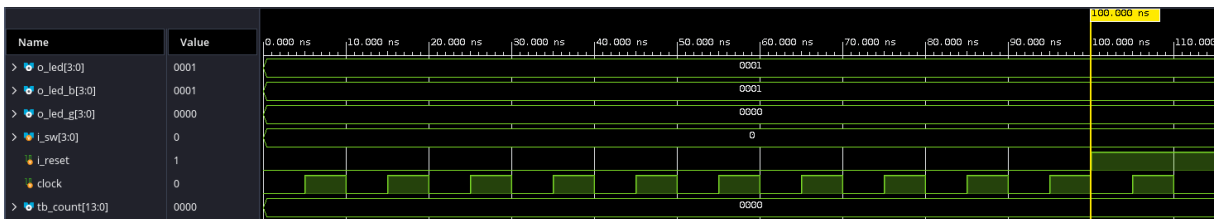


Figura 2: Simulación: reset inicial, liberado en 100 ns.

Cambios de velocidad. En las Figuras 3, 4 y 5 se ve cada cambio de `sw[2:1]` (cursor amarillo). Con los marcadores (en azul) se midió el tiempo entre dos desplazamientos consecutivos antes y después de cada cambio:

Cambio	`sw[2:1]`	Marcadores [ns]	Intervalo
1 ms	`00`	999 875 − 999 715	160 ns
	`01`	1 000 835 − 1 000 515	320 ns
2 ms	`01`	1 999 555 − 1 999 235	320 ns
	`10`	2 001 475 − 2 000 835	640 ns
4 ms	`10`	3 998 915 − 3 998 275	640 ns
	`11`	4 002 755 − 4 001 475	1280 ns

Los intervalos coinciden con los calculados (2^{4+k} ciclos de 10 ns) y cada cambio de límite duplica el tiempo entre pasos. El LED encendido sigue rotando 0001 → 0010 → 0100 → 1000 → 0001 sin perder la posición al cambiar la velocidad.

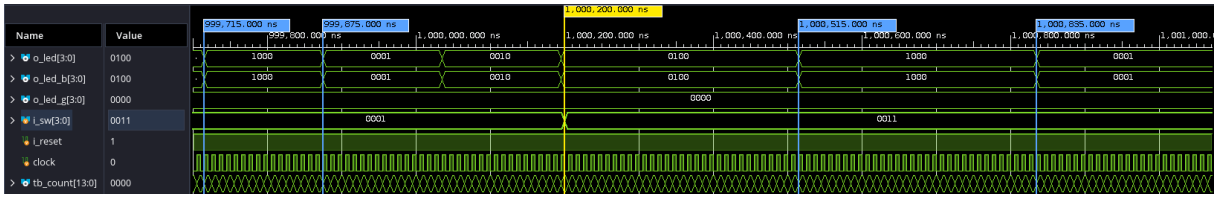


Figura 3: Simulación: cambio de `sw[2:1]` de `00` a `01` en 1 ms.

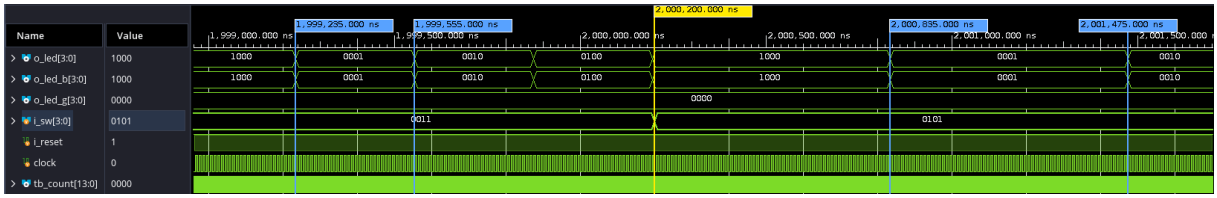


Figura 4: Simulación: cambio de `sw[2:1]` de `01` a `10` en 2 ms.

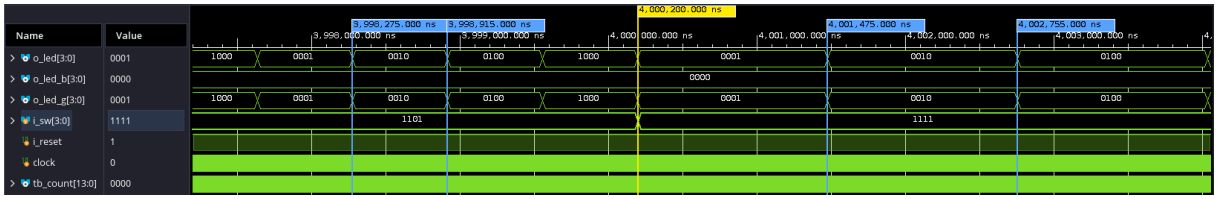


Figura 5: Simulación: cambio de `sw[2:1]` de `10` a `11` en 4 ms (con los LEDs en verde).

Cambio de color. En la Figura 6, en 3 ms `sw[3]` pasa a 1 (`i_sw`: `0101` → `1101`). En ese momento el estado `0010` pasa de `o_led_b` a `o_led_g` y el canal azul queda en `0000`. El siguiente desplazamiento (`0100`) ya aparece en verde y `o_led` no se ve afectado, porque el color solo cambia los multiplexores de salida.

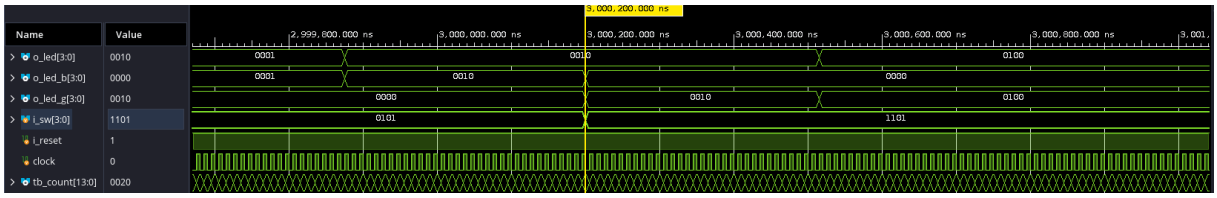


Figura 6: Simulación: cambio de color con `sw[3]` en 3 ms.

3. Implementación en la FPGA

Como no se tiene acceso físico a la placa, el diseño se implementó en una Arty A7 de un cluster remoto, y para manejarlo y observarlo se agregaron dos IP cores:

- **VIO**, con 3 entradas de 4 bits (`connect_leds`, `led_b`, `led_g`) para ver el estado, y 3 salidas: `selMux` (1 bit), el reset (1 bit, activo en bajo) y los switches (4 bits). Al pasar a `selMux = 1` hay que poner el reset del VIO en 1; si queda en 0 el sistema se mantiene en reset.
- **ILA**, con 3 probes de 4 bits conectados a las mismas señales internas, para capturar las formas de onda con el reloj del sistema.

El flujo fue: crear el proyecto en Vivado para la Arty A7 con los archivos de `src/` y `constrains/`, agregar los IP `vio` e `ila` con esa configuración, sintetizar, implementar y generar el bitstream. Después, conectarse por SSH al servidor del cluster, solicitar una FPGA, agregar en el Hardware Manager de Vivado el *remote server* con la IP asignada, y programar la placa. Luego, mediante el VIO y el ILA, se analizó el funcionamiento del diseño implementado en la placa.

3.1. VIO

En ambas capturas el VIO tiene el control del diseño (`probe_out0`, `selMux` = 1) y su reset está en 1, es decir, liberado (`probe_out1`). La salida `probe_out2` son los switches.

Con `probe_out2 = 0` (Figura 7) el sistema está deshabilitado (`sw[0] = 0`): las entradas muestran el LED detenido en 2 (`0010`) en el estado y en el canal azul, con el verde en 0, y la columna *Activity* está vacía porque nada cambia. Con `probe_out2 = 1` (Figura 8) se habilita el sistema con `sw[2:1] = 00` y color azul: el estado y el canal azul empiezan a cambiar, lo que el VIO indica con las flechas de actividad, mientras el canal verde sigue en 0.

Name	Value	Activity	Direc...	VIO
u_vio/probe_in1_0_1[3:0]	[H] 2		Input	hw_vio_1
u_vio/probe_in0_0_1[3:0]	[H] 2		Input	hw_vio_1
u_vio/probe_in2_0_1[3:0]	[H] 0		Input	hw_vio_1
u_vio/vio_0_probe_out0[0:0]	[B] 1		Output	hw_vio_1
u_vio/vio_0_probe_out1[0:0]	[B] 1		Output	hw_vio_1
u_vio/vio_0_probe_out2[3:0]	[H] 0		Output	hw_vio_1
u_vio/vio_0_probe_out2[3]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[2]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[1]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[0]	0		Output	hw_vio_1

Figura 7: VIO con el sistema deshabilitado.

Name	Value	Activity	Direc...	VIO
u_vio/probe_in1_0_1[3:0]	[H] 4		Input	hw_vio_1
u_vio/probe_in0_0_1[3:0]	[H] 4		Input	hw_vio_1
u_vio/probe_in2_0_1[3:0]	[H] 0		Input	hw_vio_1
u_vio/vio_0_probe_out0[0:0]	[B] 1		Output	hw_vio_1
u_vio/vio_0_probe_out1[0:0]	[B] 1		Output	hw_vio_1
u_vio/vio_0_probe_out2[3:0]	[H] 1		Output	hw_vio_1
u_vio/vio_0_probe_out2[3]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[2]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[1]	0		Output	hw_vio_1
u_vio/vio_0_probe_out2[0]	1		Output	hw_vio_1

Figura 8: VIO con el sistema habilitado.

3.2. ILA

El ILA guarda una ventana de 1024 muestras con el reloj de 100 MHz, unos 10 µs. Como en la placa el LED se desplaza cada ≈ 42 ms, una captura libre casi nunca mostraría un cambio. Por

eso se configuró un trigger con la condición `probe0 == 2` y `probe1 == 2`, es decir, el instante en que el estado pasa a `0010` en el canal azul. El ILA ubica el trigger en el centro de la ventana (muestra 512), así que la captura (Figura 9) muestra la transición de `1` a `2` en el estado (`probe0`) y en el canal azul (`probe1`), con el canal verde (`probe2`) en 0. Esto confirma en el hardware el mismo comportamiento que en la simulación.

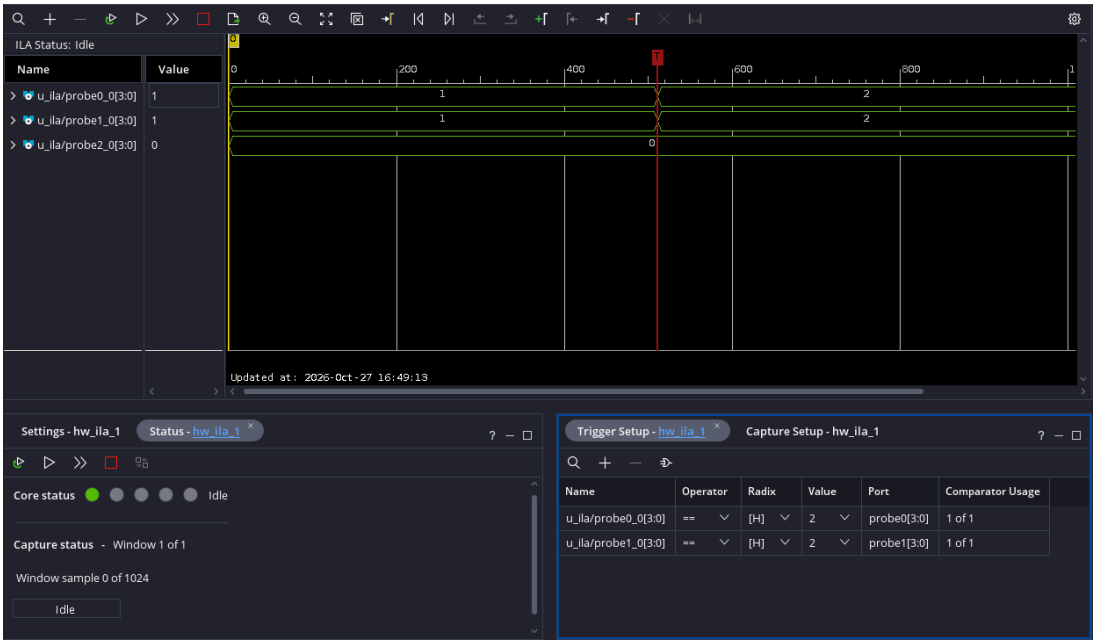


Figura 9: Captura del ILA con trigger en la transición del estado a `0010`.